```
$a = 'some val';
$b = $a;
$b = 'new val';

var_dump($a);
var_dump($b);

$bref = &$a;
$bref = 'new val';

var_dump($a);

Output:
string(8) "some val"
string(7) "new val"
string(7) "new val"
```

We set `$a` to `'some val'`, and we assign the value of `$a` to `$b`. When we change `$b`, we can see that `$a` remains unchanged. However, when `$bref` is initialized and given its value, we use an additional modifier of `&`. This ampersand character modifies the behavior of assignment: it tells PHP that `$bref` is an alias for the same *zval* as `$a`. (Note that the `&` symbol is a bit of a special case in PHP; it isn't an operator as such but changes the behavior of variables in certain contexts.)

For those of you that have some exposure to C (the language PHP is implemented in), it might be tempting to think of PHP references as equivalent to C pointers, but they are not. In PHP, when a variable is assigned in your script, the PHP interpreter creates a data structure called a *zval*. A *zval* is not directly accessible in PHP code; they are internal to the PHP engine and defined in the C source code in which PHP is written. When a variable is initialized in PHP code, the engine creates a *zval*, and the variable name is used as an alias for it. When a reference is created in PHP, the two variables are now aliases for a single *zval*.

References used as above are rare and should probably be kept that way. Use them sparingly. You are far more likely to see variable references in the context of function parameters, and we'll revisit this particular feature of PHP variables in Chapter 3, "Functions."

Things get interesting when destroying variables that are referenced. Unsetting a reference itself does what you might expect: the reference variable is no longer available, but the referred variable continues to exist: